UNIVERSITY OF JAMMU

BHADERWAH CAMPUS

BACHELOR OF COMPUTER APPLICATION

(BCA-4th SEMESTER 2024-28)

PRACTICAL FILE

PYTHON PROGRAMMING

SUBMITTED BY:

Damit Singh

413040013

SUBMITTED TO:

Mr. Abhay Khajuria

(Astt. Professor)

Dept. Of CS and IT

ACKNOWLEDGEMENT

I would like to express my gratitude to all those people who helped me to complete this practical file. I want to thank **Department of computer science and IT** for providing me all the tools and resources for the completion of my practical file.

I have further more to thank our respected teacher **Mr. Abhay Khajuria** who encouraged me to move ahead and complete my work in time.

At last, I want to thank my classmates and friends for their help, support, interest and valuable hints. I want to thank once again all the above mentioned persons for their time and help.

Thank you.

Damit Singh

BCA 4th Semester

413040013

CERTIFICATE

This is to certify that **Damit Singh** student of **BCA-4th semester** bearing university **Roll no. 413040013** has completed the required number of practicals of **Python Programming** under the guidance of class teacher **Mr. Abhay Khajuria** from the Department of Computer science and IT, Bhaderwah campus, University of Jammu.

Signature.

(I/C course coordinator)

Signature.

(Practical I/C)

Index

No.	Contents	Remarks
1	WAP to check whether a number is even or odd.	
2	WAP to check whether a given number is prime or not.	
3	WAP to find the largest among three numbers.	
4	WAP to check whether a given year is a leap year or not.	
5	WAP to calculate the factorial of a given number.	
6	WAP to generate the Fibonacci sequence up to n terms.	
7	WAP to reverse a given number without using built-in function.	
8	WAP to check whether a given number is an Armstrong number.	
9	WAP to check whether a given string is a palindrome or not.	
10	WAP to count the number of vowels and consonants in a given string.	
11	WAP to count the frequency of each character in a string.	
12	WAP to reverse a given string.	
13	WAP to remove duplicate characters from a given string.	
14	WAP to sort the character of a string in alphabetical order.	
15	WAP to find the longest word in a given sentence.	
16	WAP to find the first occurrence of a substring in a string.	
17	WAP to find the sum of all elements in a given list.	
18	WAP to find the largest and smallest elements in a given list.	
19	WAP to remove duplicate elements from a list.	
20	WAP to merge two list into one.	
21	WAP to sort a list in ascending and descending order.	
22	WAP to find common elements between two strings.	
23	WAP to check whether a list is sorted or not.	
24	WAP to rotate elements of a list by n positions.	
25	WAP to find the second largest number in a list.	
26	WAP to count the frequency of words in a given sentence using a dictionary.	

27	**WAP to check whether a key exists in a dictionary.**	
28	**WAP to sort a dictionary by its values.**	
29	**WAP to convert two lists into a dictionary (keys and values).**	
30	**WAP to perform union, intersection and difference operation on sets.**	

#Program<1>

WAP to check whether a given number is **even** or **odd**.

```
def find_even_odd(number):
    if number & 1 == 0:
        return "Even Number"
    return "Odd Number"


user_val = int(input("Provide a number: "))
print(f"The number {user_val} is an {find_even_odd(user_val)}")
```

Output:-

```
hp@Damit:~$ python prog1.py
Provide a number: 37
The number 37 is an Odd Number
```

#Program<2>

WAP to check whether a given number is **prime** or **not**.

```
def check_prime_status(val):
    if val < 2:
        return False
    for divisor in range(2, int(val ** 0.5) + 1):
        if val % divisor == 0:
            return False
    return True


num_input = int(input("Enter number to verify: "))
if check_prime_status(num_input):
    print(f"{num_input} is a Prime Number.")
else:
    print(f"{num_input} is NOT a Prime Number.")
```

Output:-

```
hp@Damit:~$ python prog2.py
Enter number to verify: 33
33 is NOT a Prime Number.
```

#Program<3>

WAP to find the **largest** among **three numbers**.

```
x = int(input("Enter first integer: "))

y = int(input("Enter second integer: "))

z = int(input("Enter third integer: "))


max_val = x

if y > max_val:

    max_val = y

if z > max_val:

    max_val = z


print(f"The maximum value is: {max_val}")
```

Output:-

```
hp@Damit:~$ python prog3.py
Enter first integer: 100
Enter second integer: 50
Enter third integer: 75
The maximum value is: 100
```

#Program<4>

WAP to check whether a given **year** is a **leap year** or **not**.

```
input_year = int(input("Enter a year: "))
if input_year % 400 == 0 or (input_year % 4 == 0 and input_year % 100 != 0):
    print("Result: Leap Year")
else:
    print("Result: Normal Year")
```

Output:-

```
hp@Damit:~$ python prog4.py
Enter a year: 2023
Result: Normal Year
```

#Program<5>

WAP to calculate the **factorial** of a **given number.**

```
n = int(input("Enter a positive number: "))

factorial_result = 1

current = n


while current > 0:

    factorial_result *= current

    current -= 1


print(f"Factorial = {factorial_result}")
```

Output:-

```
hp@Damit:~$ python prog5.py
Enter a positive number: 5
Factorial = 120
```

#Program<6>

WAP to generate the **Fibonacci sequence** up to **n** terms.

```
limit = int(input("Enter number of terms required: "))
series = [0, 1]


if limit <= 0:
    print("Please enter a positive integer.")
elif limit == 1:
    print("Series:", [0])
else:
    while len(series) < limit:
        next_val = series[-1] + series[-2]
        series.append(next_val)
    print("Series:", series)
```

Output:-

```
hp@Damit:~$ python prog6.py
Enter number of terms required: 8
Series: [0, 1, 1, 2, 3, 5, 8, 13]
```

#Program<7>

WAP to **reverse** a given number without using **built-in function**.

```
original_number = int(input("Type a number: "))
reversed_result = 0


while original_number != 0:
    digit = original_number % 10
    reversed_result = (reversed_result * 10) + digit
    original_number //= 10


print(f"Flipped Number: {reversed_result}")
```

Output:-

```
hp@Damit:~$ python prog7.py
Type a number: 13579
Flipped Number: 97531
```

#Program<8>

WAP to check whether a given number is an **Armstrong number**.

```
number_string = input("Enter a number to check: ")

length = len(number_string)

calculated_sum = 0


for char in number_string:

    calculated_sum += int(char) ** length


if str(calculated_sum) == number_string:

    print("Validation: True (It is an Armstrong number)")

else:

    print("Validation: False")
```

Output:-

```
hp@Damit:~$ python prog8.py
Enter a number to check: 370
Validation: True (It is an Armstrong number)
```

#Program<9>

WAP to check whether a given **string** is a **palindrome** or not.

```
user_str = input("Type a word: ")
is_palindrome = True

for i in range(len(user_str) // 2):
    if user_str[i] != user_str[-(i + 1)]:
        is_palindrome = False
        break

if is_palindrome:
    print("The word is a palindrome.")
else:
    print("The word is not a palindrome.")
```

Output:-

```
hp@Damit:~$ python prog9.py
Type a word: level
The word is a palindrome.
```

#Program<10>

WAP to count the number of **vowels** and **consonants** in a given **string**.

```
sentence = input("Type a sentence: ").replace(" ", "").lower()

vowels_list = {'a', 'e', 'i', 'o', 'u'}

v_count = 0

c_count = 0


for alphabet in sentence:

    if alphabet.isalpha():

        if alphabet in vowels_list:

            v_count += 1

        else:

            c_count += 1


print(f"Total Vowels = {v_count}")

print(f"Total Consonants = {c_count}")
```

Output:-

```
hp@Damit:~$ python prog10.py
Type a sentence: coding is awesome
Total Vowels = 7
Total Consonants = 8
```

#Program<11>

WAP to count the **frequency** of each **character** in a **string**.

```
target_string = input("Provide text: ")

character_dict = {}


for letter in target_string:

    if letter in character_dict:

        character_dict[letter] += 1

    else:

        character_dict[letter] = 1


print("Frequencies:", character_dict)
```

Output:-

```
hp@Damit:~$ python prog11.py
Provide text: hello
Frequencies: {'h': 1, 'e': 1, 'l': 2, 'o': 1}
```

#Program<12>

WAP to **reverse** a given **string**.

```
string_to_reverse = input("String input: ")

backward_str = ""


for char in string_to_reverse:

    backward_str = char + backward_str


print(f"Reversed string format: {backward_str}")
```

Output:-

```
hp@Damit:~$ python prog12.py
String input: thereby
Reversed string format: ybereht
```

#Program<13>

WAP to **remove duplicate characters** from a given **string**.

```
input_word = input("Enter a string: ")

no_duplicates = "".join(dict.fromkeys(input_word))

print(f"Cleaned string: {no_duplicates}")
```

Output:-

```
hp@Damit:~$ python prog13.py
Enter a string: programming
Cleaned string: progamin
```

#Program<14>

WAP to **sort** the character of a **string** in **alphabetical** order.

```
jumbled_string = input("Enter jumbled string: ")

character_list = list(jumbled_string)

character_list.sort()

print("Alphabetical order:", "".join(character_list))
```

Output:-

```
hp@Damit:~$ python prog14.py
Enter jumbled string: python
Alphabetical order: hnopty
```

#Program<15>

WAP to find the **longest** word in a given **sentence**.

```
text_line = input("Provide a full sentence: ")

word_array = text_line.split()

max_word = ""


for item in word_array:

    if len(item) > len(max_word):

        max_word = item


print(f"The longest word extracted: {max_word}")
```

Output:-

```
hp@Damit:~$ python prog15.py
Provide a full sentence: I love to study data structures
The longest word extracted: structures
```

#Program<16>

WAP to find the **first occurrence** of a **substring** in a string.

```
base_text = input("Enter main text: ")

search_term = input("Enter substring: ")


try:

    loc = base_text.index(search_term)

    print(f"Substring starts at index number: {loc}")

except ValueError:

    print("Substring does not exist in the text.")
```

Output:-

```
hp@Damit:~$ python prog16.py
Enter main text: she sells sea shells
Enter substring: sea
Substring starts at index number: 10
```

#Program<17>

WAP to find the **sum** of all **elements** in a given **list**.

```
size = int(input("Number of values in list: "))

collection = []

total_sum = 0


for i in range(size):

    val = int(input(f"Value [{i}]: "))

    collection.append(val)

    total_sum += val


print(f"List sum evaluates to: {total_sum}")
```

Output:-

```
hp@Damit:~$ python prog17.py
Number of values in list: 4
Value [0]: 5
Value [1]: 5
Value [2]: 5
Value [3]: 5
List sum evaluates to: 20
```

#Program<18>

WAP to find the **largest** and **smallest** elements in a given **list**.

```
num_elements = int(input("Enter count: "))
data_list = []

for _ in range(num_elements):
    data_list.append(int(input("Enter number: ")))

max_val = data_list[0]
min_val = data_list[0]

for number in data_list:
    if number > max_val:
        max_val = number
    if number < min_val:
        min_val = number

print(f"Maximum = {max_val} | Minimum = {min_val}")
```

Output:-

```
hp@Damit:~$ python prog18.py
Enter count: 4
Enter number: 12
Enter number: 5
Enter number: 89
Enter number: 1
Maximum = 89 | Minimum = 1
```

#Program<19>

WAP to remove **duplicate** elements from a **list**.

```
raw_entries = input("Provide space-separated values: ").split()

filtered_entries = []


[filtered_entries.append(x) for x in raw_entries if x not in filtered_entries]


print(f"List without duplicates: {filtered_entries}")
```

Output:-

```
hp@Damit:~$ python prog19.py
Provide space-separated values: a b a c b d
List without duplicates: ['a', 'b', 'c', 'd']
```

#Program<20>

WAP to **merge** two **list** into **one**.

```
first_list = input("Input list A: ").split()

second_list = input("Input list B: ").split()


first_list.extend(second_list)

print("Final combined list:", first_list)
```

Output:-

```
hp@Damit:~$ python prog20.py
Input list A: 1 2
Input list B: 3 4
Final combined list: ['1', '2', '3', '4']
```

#Program<21>

WAP to **sort** a list in **ascending** and **descending** order.

```
values = [int(v) for v in input("Enter integers to sort: ").split()]


values.sort()

print(f"Ascending order: {values}")


values.sort(reverse=True)

print(f"Descending order: {values}")
```

Output:-

```
hp@Damit:~$ python prog21.py
Enter integers to sort: 9 1 4 7
Ascending order: [1, 4, 7, 9]
Descending order: [9, 7, 4, 1]
```

#Program<22>

WAP to find **common** elements between **two strings**.

```
s1 = input("Type string one: ")

s2 = input("Type string two: ")


common_chars = "".join(set([char for char in s1 if char in s2]))

print(f"Intersecting characters: {common_chars}")
```

Output:-

```
hp@Damit:~$ python prog22.py
Type string one: earth
Type string two: heart
Intersecting characters: aehrt
```

#Program<23>

WAP to check whether a list is **sorted** or **not**.

```
num_list = [int(x) for x in input("Enter numbers: ").split()]


is_asc = all(num_list[i] <= num_list[i + 1] for i in range(len(num_list) - 1))
is_desc = all(num_list[i] >= num_list[i + 1] for i in range(len(num_list) - 1))


if is_asc or is_desc:
    print("The list is currently sorted.")
else:
    print("The list is unsorted.")
```

Output:-

```
hp@Damit:~$ python prog23.py
Enter numbers: 9 7 5 3
The list is currently sorted.
```

#Program<24>

WAP to **rotate** elements of a list by **n positions**.

```
data_arr = input("Enter items: ").split()

rotations = int(input("How many steps to rotate right? "))


rotations = rotations % len(data_arr)

shifted_arr = data_arr[-rotations:] + data_arr[:-rotations]

print(f"After rotation: {shifted_arr}")
```

Output:-

```
hp@Damit:~$ python prog24.py
Enter items: 1 2 3 4 5
How many steps to rotate right? 3
After rotation: ['3', '4', '5', '1', '2']
```

#Program<25>

WAP to find the **second largest** number in a list.

```
number_series = [float(x) for x in input("Enter numbers: ").split()]


if len(set(number_series)) < 2:
    print("Not enough unique numbers.")
else:
    number_series.remove(max(number_series))
    print(f"The second largest element is: {max(number_series)}")
```

Output:-

```
hp@Damit:~$ python prog25.py
Enter numbers: 100 200 50 150
The second largest element is: 150.0
```

#Program<26>

WAP to count the **frequency** of words in a given sentence using a **dictionary**.

```
phrase = input("Enter a sentence: ").lower().split()

word_tracker = {}


for w in phrase:

    if w in word_tracker:

        word_tracker[w] += 1

    else:

        word_tracker[w] = 1


print("Word Dictionary Mapping:", word_tracker)
```

Output:-

```
hp@Damit:~$ python prog26.py
Enter a sentence: study hard play hard
Word Dictionary Mapping: {'study': 1, 'hard': 2, 'play': 1}
```

#Program<27>

WAP to check whether a **key exists** in a **dictionary**.

```
inventory = {"apples": 10, "bananas": 5, "oranges": 8}

item_to_check = input("Enter item to search for: ")


if item_to_check in inventory.keys():

    print(f"Found it! {item_to_check} is in the inventory.")

else:

    print(f"Missing! {item_to_check} is not listed.")
```

Output:-

```
hp@Damit:~$ python prog27.py
Enter item to search for: grapes
Missing! grapes is not listed.
```

#Program<28>

WAP to **sort** a **dictionary** by its **values**.

```
raw_dict = {"Math": 85, "Science": 92, "English": 78}

sorted_by_val = {k: v for k, v in sorted(raw_dict.items(), key=lambda item: item[1])}

print(f"Ascending Value Sort: {sorted_by_val}")
```

Output:-

```
hp@Damit:~$ python prog28.py
Ascending Value Sort: {'English': 78, 'Math': 85, 'Science': 92}
```

#Program<29>

WAP to convert two **lists** into a **dictionary** (**keys** and **values**).

```
attr_list = input("Enter attributes (keys): ").split()

val_list = input("Enter details (values): ").split()


constructed_dict = {}

for idx in range(min(len(attr_list), len(val_list))):

    constructed_dict[attr_list[idx]] = val_list[idx]


print("Created Dictionary:", constructed_dict)
```

Output:-

```
hp@Damit:~$ python prog29.py
Enter attributes (keys): city state
Enter details (values): Jammu JK
Created Dictionary: {'city': 'Jammu', 'state': 'JK'}
```

#Program<30>

WAP to perform **union**, **intersection** and **difference** operation on **sets**.

```
set_x = {10, 20, 30, 40}

set_y = {30, 40, 50, 60}


print(f"Set X: {set_x} | Set Y: {set_y}")

print(f"Union (X | Y): {set_x.union(set_y)}")

print(f"Intersection (X & Y): {set_x.intersection(set_y)}")

print(f"Difference (X - Y): {set_x.difference(set_y)}")
```

Output:-

```
hp@Damit:~$ python prog30.py
Set X: {40, 10, 20, 30} | Set Y: {40, 50, 60, 30}
Union (X | Y): {40, 10, 50, 20, 60, 30}
Intersection (X & Y): {40, 30}
Difference (X - Y): {10, 20}
```